

Hamro News CMS - Source Reference Guide

A step-by-step git roadmap corresponding to the Web Technology course syllabus

🎓 Pedagogical Strategy

This repository uses an evolutionary Git strategy. Instead of dropping a massive complete codebase on day one, students follow specific milestone branches. Each branch isolates the core programming concepts required by individual syllabus blocks—moving progressively from **Procedural Scripts** to **Encapsulated OOP Objects**, **Filesystem CRUD**, **Relational SQL**, and finally **Secure Session Gateways**.

branch: main

UNIT 1 & UNIT 2 INTRO BASE

Syllabus Focus: Server-Side Scripting, Base Syntax, Includes/Requires, Absolute Path Configurations.

📁 Architectural File Layout Structure

```
news/
├── config/
│   └── config.php
├── public/
│   ├── index.php
│   └── dashboard.php
├── templates/
├── header.php
└── footer.php
```

🔑 Core Innovations & Learning Objectives

- Established absolute file system routing rules inside `config.php` using `dirname(__DIR__)` to completely avoid nested directory file breakdown errors (e.g., `SRC_PATH`, `TEMPLATE_PATH`).
- Taught template splitting layout mechanics using `require_once` across entry points.

branch: 01-mock-data-array

UNIT 2: STRINGS & ARRAYS

Syllabus Focus: Associative Arrays, Multidimensional Data Arrays, `foreach` Rendering Loops, URL Parameter Capture (`$_GET`).

📁 Architectural File Layout Structure

```
news/
├── public/
│   ├── index.php
│   └── blog-view.php
├── src/
├── database/
└── mock_posts.php
```

📁 Core Innovations & Learning Objectives

- Students transition away from manual layout duplication by learning to loop through a multi-dimensional associative array.
- Introduces `htmlspecialchars()` and `urlencode()` to safeguard outputs against XSS injections and stabilize data passing via parameters.

branch: 02-oop-architecture

UNIT 3: OBJECT-ORIENTED PROGRAMMING

Syllabus Focus: Encapsulation, Visibility Modifiers, Getter Methods, Interfaces (Contracts), Namespaces, Static Factory Methods.

📁 Architectural File Layout Structure

```
news/
├── public/
│   ├── index.php
│   ├── blog-view.php
│   └── dashboard.php
├── src/
├── database/
├── postmanager.php
├── models/
├── postinterface.php
└── post.php
```

📁 Core Innovations & Learning Objectives

- **Encapsulation:** All variables are marked `private` to prevent external data tampering, forcing fields to be read cleanly via public getters like `$post->getTitle()`.
- **Separation of Concerns:** Isolates the mapping logic away from display scripts by introducing a central data handler class: `PostManager`.
- **Namespacing:** Prevents namespace collision risks using strict PSR-inspired `HamroNews\Models` and `HamroNews\Database` pathways.

branch: 03-file-and-form-handling

UNIT 4: FILE & FORM HANDLING

Syllabus Focus: HTML Forms, `$_POST` Form Processing, File Streaming (`fopen`, `fgets`, `file_put_contents`), Serialization via JSON, Flat-File CRUD Engine.

📁 Architectural File Layout Structure

```
news/
├── public/
│   ├── dashboard.php
│   └── edit-post.php
├── src/
├── database/
└── postmanager.php
```

```
├── user_submitted_posts.txt
├── create_post.php
├── update_post_processor.php
└── delete_post_processor.php
```

Core Innovations & Learning Objectives

- **Pre-seeded Storage Architecture:** Pre-populating `user_submitted_posts.txt` with 6 JSON lines demonstrates real-world parsing immediately, making clear how `json_encode()` and `json_decode()` manage serialization.
- **In-Memory Mutation Filters:** Teaches file system data manipulation under pressure by loading full documents into memory array logs via `file()`, mutating specific rows, and securely overwriting disk data using exclusive thread locks (`LOCK_EX`).

branch: 04-database-integration

UNIT 5: RELATIONAL DATABASES & PDO

Syllabus Focus: SQL Schema Scripting, PHP Data Objects (PDO) Architecture, Centralized Connection Credentials, Parameterized Prepared Statements, SQL CRUD Integration.

Architectural File Layout Structure

```
news/
├── config/
│   └── config.php
├── public/
│   ├── index.php
│   └── dashboard.php
├── src/
├── database/
├── schema.sql
├── db.php
├── postmanager.php
├── create_post.php
├── update_post_processor.php
└── delete_post_processor.php
```

Core Innovations & Learning Objectives

- **The Architectural "Aha!" Moment:** Demonstrates the power of clean Object-Oriented isolation. Because the frontend tracks were tied exclusively to the `PostManager` interface definitions from Unit 3, switching from flat-text storage to a full MySQL cluster required modifying **zero lines** of code across the frontend files (`index.php`, `blog-view.php`).
- **SQL Injection Defense:** Eradicates data vulnerability concerns by replacing dangerous direct SQL queries with parameterized prepared statements (e.g., `:title`, `:slug`), ensuring user inputs are handled exclusively as isolated variables, never executable query code.

branch: 05-cookies-sessions-auth

UNIT 6: COOKIES, SESSIONS & SECURITY

Syllabus Focus: State Persistence, Session Lifecycle Tracking, Defensive Programming Notices, Cryptographic Password Hashing, Access Guard Authorization Layers.

📁 Architectural File Layout Structure

```
news/
├── public/
│   ├── login.php
│   ├── logout.php
│   ├── dashboard.php
│   └── edit-post.php
├── templates/
└── header.php
```

📌 Core Innovations & Learning Objectives

- **Why Sessions Were Introduced:** HTTP is completely stateless—every single page request is an independent transaction with no memory of prior interactions. Sessions fix this by maintaining user state. When an admin logs in, a unique cookie token is passed to their browser, allowing the server to recognize them and keep administrative pages locked away from unauthenticated guests.
- **Defensive State Management:** Solves standard session collision bugs using strict status checking logic: `if (session_status() === PHP_SESSION_NONE) { session_start(); }`. This pattern prevents duplicate initialization crashes and avoids the "Headers already sent" runtime execution warnings.
- **Secure Cryptographic Storage:** Introduces modern password security. Instead of storing dangerous raw text parameters, users are checked against secure hashes via `password_hash()` and evaluated safely with `password_verify()` using native BCrypt configurations.